

Que aula de mierda (otra vez)
Che sabes que me gustó Xournal

¿I.A.?

Vamos a hacer un lavatropas
con I.A.

Regresión $\rightarrow$ Predicir Y con una
observación de X

El algoritmo aprende

Taller de Procesamiento de Señales / Introducción a la Inteligencia Artificial

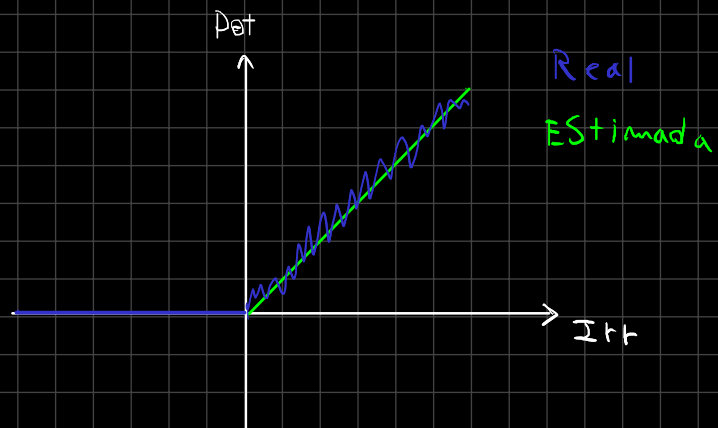
Jetga Marketimeta

No hay q' tomarla tan literal

Agenda

- 1 Introducción al problema de regresión
- 2 Regresión Lineal
- 3 Gradiente Descendente
- 4 Regresión Polinómica

Sobre el TP01:



Teoría de Regresión

Bases

Objetivo: Predecir el valor de Y a partir de $X \rightarrow \hat{Y} = \varphi(X)$

Función costo: Error cuadrático $\rightarrow \ell(x, y) = (y - \varphi(x))^2$

Riesgo Esperado: MSE $\rightarrow \mathbb{E}[\ell(X, Y)] = \mathbb{E}[(Y - \varphi(X))^2]$

Teoría de Regresión

Bases

Objetivo: Predecir el valor de Y a partir de $X \rightarrow \hat{Y} = \varphi(X)$

Función costo: Error cuadrático $\rightarrow \ell(x, y) = (y - \varphi(x))^2$

Riesgo Esperado: MSE $\rightarrow \mathbb{E}[\ell(X, Y)] = \mathbb{E}[(Y - \varphi(X))^2]$

Optimalidad

$$\mathbb{E}[(Y - \varphi(X))^2] \geq \mathbb{E}[\text{var}(Y|X)]$$

con igualdad si y solo si $\varphi(x) = \mathbb{E}[Y|X = x]$.

Regresor óptimo: $\varphi(x) = \mathbb{E}[Y|X = x]$

Error Bayesiano: $\mathbb{E}[\text{var}(Y|X)] \rightarrow$ Lo de la guía

↓
Mínimo error q' puede alcanzar

Veamos por qué el error bayesiano es así cuadrado

$$E[(Y - \phi(X))^2] = E[(\underbrace{(Y - E[Y|X])}_{\text{I}}) + (\underbrace{E[Y|X] - \phi(X)}_{\text{II}})]^2 = \underbrace{E[(Y - E[Y|X])^2]}_{\text{I}} + \underbrace{E[(E[Y|X] - \phi(X))^2]}_{\text{II}} + 2 \underbrace{E[(Y - E[Y|X])(E[Y|X] - \phi(X))]}_{\text{III}}$$

I $E[(Y - E[Y|X])^2] = E[E[(Y - E[Y|X])^2 | X]] = \text{Var}(Y|X)$

II $E[(E[Y|X] - \phi(X))^2] \geq 0$ con = si; $\phi(X) = E[Y|X]$

Está en el asunto
en sí

III $E[(Y - E[Y|X])(E[Y|X] - \phi(X))] = E[E[(Y - E[Y|X])(E[Y|X] - \phi(X)) | X]] = \text{No llegó} = 0$

La cuestión es que el menor error que puedo tener es $\text{var}[Y|X]$, con igualdad cuando $\phi(X)$ sea $E[Y|X]$

Parece que la I.A. es básicamente esto

Reconocimiento de patrones

Objetivo

Quiero buscar $\varphi(\cdot)$ que minimice $\mathbb{E}[\ell(X, Y)]$. Es decir aprender la “esperanza condicional”.

Reconocimiento de patrones

Objetivo

Quiero buscar $\varphi(\cdot)$ que minimice $\mathbb{E}[\ell(X, Y)]$. Es decir aprender la “esperanza condicional”.

Empirical Risk Minimization (ERM)

Propongo buscar $\varphi(\cdot)$ que minimice el riesgo empírico: $\frac{1}{n} \sum_{i=1}^n \ell(X_i, Y_i)$

Reconocimiento de patrones

Objetivo

Quiero buscar $\varphi(\cdot)$ que minimice $\mathbb{E}[\ell(X, Y)]$. Es decir aprender la “esperanza condicional”.

Empirical Risk Minimization (ERM) *ECM promedio de las muestras* $\leftarrow (y_i - \varphi(x_i))^2$

Propongo buscar $\varphi(\cdot)$ que minimice el riesgo empírico: $\frac{1}{n} \sum_{i=1}^n \ell(X_i, Y_i)$

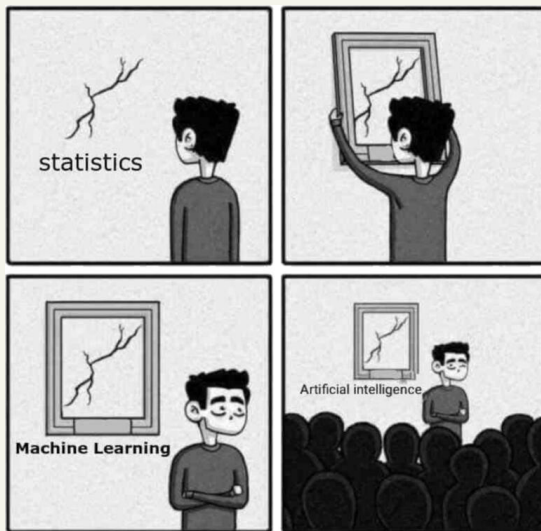
Tradeoff: Sesgo/Varianza

$$\underbrace{\mathbb{E}[\ell(X, Y)]}_{\text{Riesgo esperado}} = \underbrace{\frac{1}{n} \sum_{i=1}^n \ell(X_i, Y_i)}_{\text{Riesgo empírico}} + \underbrace{\left(\mathbb{E}[\ell(X, Y)] - \frac{1}{n} \sum_{i=1}^n \ell(X_i, Y_i) \right)}_{\text{Gap de generalización}}$$

Este es el riesgo real

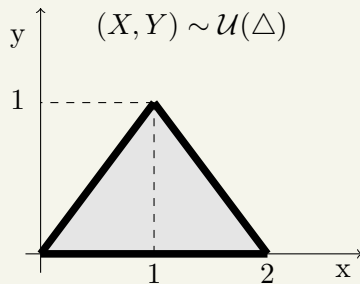
Nota: El riesgo empírico se considera grande o pequeño comparándolo con el error bayesiano.

¿Que es la Inteligencia Artificial?

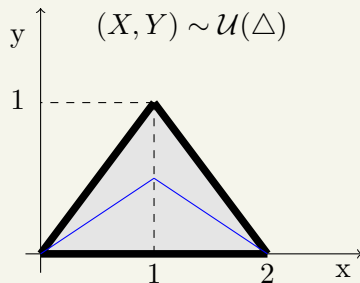


→ ES TODO HUMO

Overfitting y Underfitting



Overfitting y Underfitting



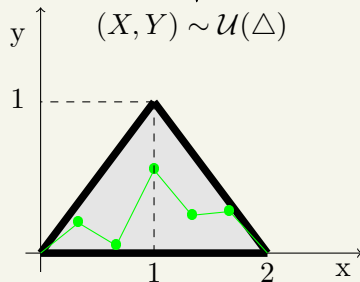
Solución Óptima

- El regresor elegido es efectivamente la esperanza condicional.
- El riesgo esperado alcanza el límite bayesiano

$$\mathbb{E} [\text{var}(Y|X)] = \frac{1}{24}$$

Overfitting y Underfitting

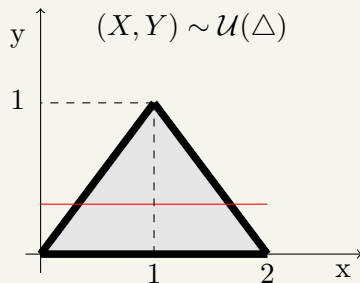
Normalmente no sabemos
cómo es la distribución



Problema de overfitting

- Riesgo empírico muy bajo (puede ser menor incluso que el bayesiano)
- Se detecta por el alto gap de generalización.
- Exceso de complejidad en el modelado.
- Se dice que el algoritmo tiene un problema de varianza.

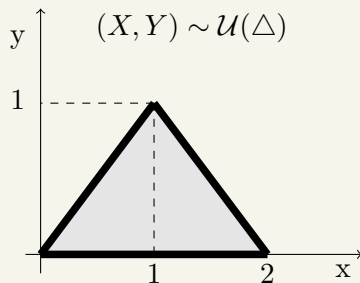
Overfitting y Underfitting



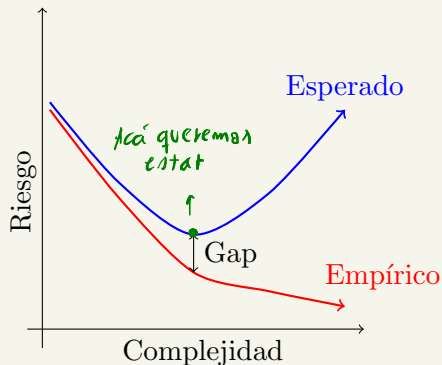
Problema de underfitting

- Suele tener bajo gap de generalización.
- Riesgo empírico muy superior al error bayesiano.
- Escasez de complejidad en el modelado.
- Se dice que el algoritmo tiene un problema de sesgo.

Overfitting y Underfitting



Teoría Clásica de Generalización



Problemas básicos del aprendizaje estadístico

Sobreajuste



Subajuste



Regresión Lineal: $\hat{Y} = w^T \cdot X + b$

Idea

Me aseguro mantener acotado el problema de overfitting proponiendo una solución de extremadamente baja complejidad. Si se alcanza bajo error empírico, entonces tengo ciertas garantías de que el algoritmo alcanza un buen desempeño.

Regresión Lineal: $\hat{Y} = w^T \cdot X + b$

Idea

Me aseguro mantener acotado el problema de overfitting proponiendo una solución de extremadamente baja complejidad. Si se alcanza bajo error empírico, entonces tengo ciertas garantías de que el algoritmo alcanza un buen desempeño.

Empirical Risk Minimization

$$(w, b) \in \arg \min_{(w, b)} \sum_{i=1}^n (w^T \cdot X_i + b - Y_i)^2$$

Problema: hallar w y b que minimicen ese ECM

Regresión Lineal: $\hat{Y} = w^T \cdot X + b$

Empirical Risk Minimization

$$(w, b) \in \arg \min_{\mathbf{w}} \|\mathbf{X} \cdot \mathbf{w} - \mathbf{y}\|^2,$$

$$\mathbf{X} = \begin{pmatrix} 1 & X_1^T \\ 1 & X_2^T \\ \vdots & \vdots \\ 1 & X_n^T \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} Y_1 \\ Y_2 \\ \vdots \\ Y_n \end{pmatrix}, \quad \mathbf{w} = \begin{pmatrix} b \\ w \end{pmatrix}$$

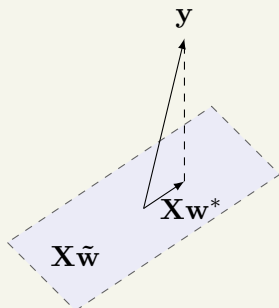
$$\vec{X} \cdot \vec{w} = \begin{pmatrix} 1 & x_1^T \\ 1 & x_2^T \\ \vdots & \vdots \\ 1 & x_m^T \end{pmatrix} \begin{pmatrix} b \\ w \end{pmatrix} = \begin{pmatrix} b + x_1^T \cdot w \\ b + x_2^T \cdot w \\ \vdots \\ b + x_m^T \cdot w \end{pmatrix}$$

Regresión Lineal: $\hat{Y} = w^T \cdot X + b$

Empirical Risk Minimization

$$(w, b) \in \arg \min_{\mathbf{w}} \|\mathbf{X} \cdot \mathbf{w} - \mathbf{y}\|^2,$$

$$\mathbf{X} = \begin{pmatrix} 1 & X_1^T \\ 1 & X_2^T \\ \vdots & \vdots \\ 1 & X_n^T \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} Y_1 \\ Y_2 \\ \vdots \\ Y_n \end{pmatrix}, \quad \mathbf{w} = \begin{pmatrix} b \\ w \end{pmatrix}$$



$$\mathbf{w}^* = \mathbf{X}^\dagger \mathbf{y}$$

Regresión Lineal

Solución matricial óptima: Recta de Regresión

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Necesitamos la pseudo inversa xq la matriz no es cuadrada

Regresión Lineal

Solución matricial óptima: Recta de Regresión

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Derivadas Matriciales

$$\nabla(\mathbf{x}^T \mathbf{a}) = \nabla(\mathbf{a}^T \mathbf{x}) = \mathbf{a}$$

$$\nabla(\mathbf{x}^T \mathbf{B} \mathbf{x}) = (\mathbf{B} + \mathbf{B}^T) \mathbf{x}$$

$$\mathcal{H}(\mathbf{x}^T \mathbf{B} \mathbf{x}) = \mathbf{B} + \mathbf{B}^T$$

¿Cual es el gradiente de $J(\mathbf{w}) = \frac{1}{n} \|\mathbf{X} \cdot \mathbf{w} - \mathbf{y}\|^2$?

Regresión Lineal

Solución matricial óptima: Recta de Regresión

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Derivadas Matriciales

$\textcircled{\text{I}} \nabla(\mathbf{x}^T \mathbf{a}) = \nabla(\mathbf{a}^T \mathbf{x}) = \mathbf{a}$ *Pensar q es un escalar*

$\textcircled{\text{II}} \nabla(\mathbf{x}^T \mathbf{B} \mathbf{x}) = (\mathbf{B} + \mathbf{B}^T) \mathbf{x}$

Hessiano (derivada
segunda)

$\leftarrow \mathcal{H}(\mathbf{x}^T \mathbf{B} \mathbf{x}) = \mathbf{B} + \mathbf{B}^T$ $\textcircled{\text{III}}$

¿Cual es el gradiente de $J(\mathbf{w}) = \frac{1}{n} \|\mathbf{X} \cdot \mathbf{w} - \mathbf{y}\|^2$?

Optimización convexa

El problema de regresión lineal es un problema convexo.

Petersen and Pedersen - "Matrix Cookbook".

$$J(w) = \frac{1}{n} (Xw - y)^T (Xw - y) = \frac{1}{n} (w^T X^T X w - 2y^T X w + y^T y)$$

$$\nabla J(w) = \frac{1}{n} (2X^T X w - 2 \overset{X^T y}{\cancel{y^T X}} + 0) = 0 \rightarrow w = (X^T X)^{-1} X^T y$$

$\downarrow$ Pot II con $B = X^T X$ $\downarrow$ Pot I $\rightarrow$ no depende de w

En la misma de esta

OBS $X^T X$ es cuadrada y (casi) siempre invertible

Hessiano: $H\{J(w)\} = \frac{2}{n} X^T X \gg 0 \rightarrow X^T X$ siempre es DP

La parábola es siempre

Siempre hay una solución

El problema es convexo

La etapa de entrenamiento es, con una cantidad de muestras de (X, Y) calcular w con esta fórmula.

Luego, las predicciones serán:

$$\hat{y}(x) = w \cdot x + b \quad (x \text{ es una observación de } X)$$

C. AUX: $(Xw - y)^T (Xw - y) = (Xw)^T - y^T (Xw - y) = w^T X^T X w + y^T y - \overset{w^T X^T y}{\cancel{w^T X^T y}} - \cancel{y^T X w} =$

$$= w^T X^T X w + y^T y - 2w^T X^T y = J(w)$$

$$\Leftrightarrow \nabla J(w) = (X^T X + [X^T X]^T) w + 0 - 2X^T y = 2X^T X w - 2X^T y$$

AHÍ TENÉS

$$\begin{array}{lcl}
 y^T \cdot X & \rightarrow & 1 \times 2 \\
 \downarrow & & \downarrow \\
 1 \times 500 & 500 \times 2 & \\
 [y^T \cdot X] \cdot w & \rightarrow & \text{igual a } 1 \times 2 \cdot 2 \times 1 \\
 \downarrow & & \downarrow \\
 1 \times 2 & 2 \times 1 & \\
 [y^T X w]^T & = & w^T X^T y
 \end{array}$$

$\nearrow$ igual a 1×2 transpuesta

Codificación de Variables Categóricas

Variables Categóricas Ordinales → Hay orden

Las variables con una clara relación de orden se las codifica con números enteros. Ej: Talles de ropa ($XS < S < M < L < XL < XXL$), jerarquía en un organigrama (Empleado junior < Empleado senior < Jefe de sector < Gerente < Director), calidad de un servicio (Deficiente < Regular < Bueno < Muy bueno < Excelente).

Variables Categóricas Nominales → No hay orden

Las variables sin una clara relación de orden se las codifica como *One-Hot*. Ej: Tipo de sangre (A, B, AB, O), religión (Ateo, Católica, Judía, Musulmana, Budista, etc.), marca de celular (Samsung, Apple, Motorola, Xiaomi, etc.).

Codificación One-Hot → Las realizaciones de la variable son un vector
Sangre B → (0, 1, 0, 0). Todos los símbolos válidos están a la misma distancia. Se fija durante el entrenamiento. → Así, todas las realizaciones "valem" lo mismo

Outline

→ Podría pasar q' haya una realización no contemplada
En ese caso se debería asignar $(1, 1, 1, 1)$
Equidista a todo el resto de realizaciones

1 Introducción al problema de regresión

2 Regresión Lineal

3 Gradiente Descendente

4 Regresión Polinómica

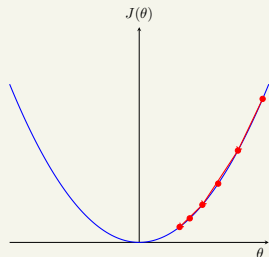
Gradiente Descendente

Problema a resolver: $\min_{\theta \in \Theta} J(\theta)$.

Solución:

$\theta \in \Theta \rightarrow$ el θ que minimiza J

$$\theta_{t+1} = \theta_t - \alpha \nabla J(\theta_t)$$



Puede ser caro hacer la inversa

$$\text{de } W = (X^T X)^{-1} X^T y$$

Decimos que α es un hiperparámetro. Se fija antes del entrenamiento y se hace tanteando

Cauchy 1847: “Méthode générale pour la résolution de systèmes d’équations simultanées”. $\rightarrow$ ¿La G.O.A.T.?

Wolpert - MacReady 1997: “No free lunch theorems for optimization”.

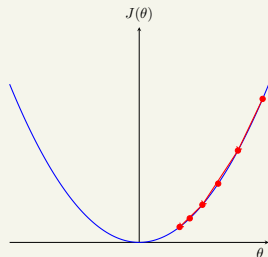
Gradiente Descendente

Problema a resolver: $\min_{\theta \in \Theta} J(\theta)$.

Solución:

$$\theta_{t+1} = \theta_t - \alpha \nabla J(\theta_t)$$

- Si α es chico la convergencia es lenta.
- Si α es grande puede no converger.



Cauchy 1847: “Méthode générale pour la résolution de systèmes d’équations simultanées”.

Wolpert - MacReady 1997: “No free lunch theorems for optimization”.

Gradiente Descendente

Problema a resolver: $\min_{\theta \in \Theta} J(\theta)$.

Solución: *learning rate*

$$\theta_{t+1} = \theta_t - \alpha \nabla J(\theta_t)$$

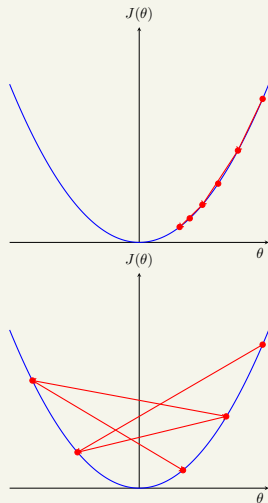
- Si α es chico la convergencia es lenta.
- Si α es grande puede no converger.

No Free Lunch

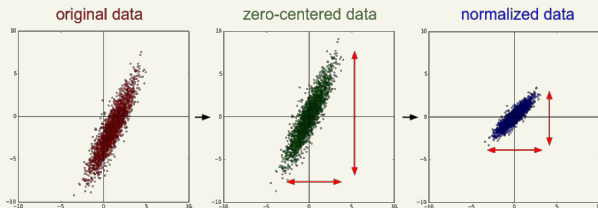
No existe un optimizador universal, depende de la tarea y los datos.

Cauchy 1847: “Méthode générale pour la résolution de systèmes d'équations simultanées”.

Wolpert - MacReady 1997: “No free lunch theorems for optimization”.



Normalización de la entrada



Normalizar *cada componente* de la entrada tiene sus beneficios:

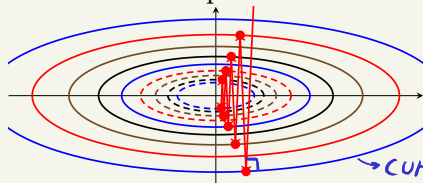
$$(\mathbf{x})_k \leftarrow \frac{(\mathbf{x})_k - \mu_k}{\sigma_k}$$

donde las μ_k y σ_k son 'calculadas previo al entrenamiento como:

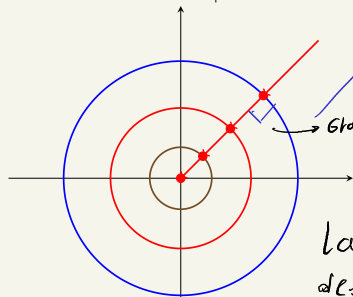
$$\mu_k = \frac{1}{n_{\text{tr}}} \sum_{i=1}^{n_{\text{tr}}} (\mathbf{x}_i)_k, \quad \sigma_k = \sqrt{\frac{1}{n_{\text{tr}}} \sum_{i=1}^{n_{\text{tr}}} [(\mathbf{x}_i)_k - \mu_k]^2}$$

Normalización de la entrada

Normalizar me permite usar learning rates más grandes!



SIN NORMALIZAR



Curvas de nivel

Gradiente $\perp$ a las curvas de nivel, va directo al centro

NORMALIZANDO

La normalización ayuda al gradiente descendente

tiene q' ver con q' el gradiente es perpendicular a las curvas de nivel

¿Cuándo y por qué normalizar?

Normalizar sí!

- Cuando quiero comparar magnitudes que por sí solas no lo son.
- Cuando quiero corregir problemas de convergencia de los algoritmos.
- Cuando el algoritmo a utilizar, necesita la hipótesis de entradas normalizadas en su génesis.

Normalizar no!

Normalizar por las dudas o por costumbre es una mala práctica.

¿Cuándo y por qué normalizar?

Normalizar si!

- Cuando quiero comparar magnitudes que por sí solas no lo son.
- Cuando quiero corregir problemas de convergencia de los algoritmos.
- Cuando el algoritmo a utilizar, necesita la hipótesis de entradas normalizadas en su génesis.

Normalizar no!

Normalizar por las dudas o por costumbre es una mala práctica.

¿Y si alguna varianza da cero?

Si algún $\sigma_k = 0$, significa que esa componente de la entrada es constante a lo largo de todo el conjunto de datos, y por lo tanto puede excluirse del análisis.

Convergencia y optimización para problemas convexos

Modelo convexo con derivadas segunda continuas

- $\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \nabla J(\mathbf{w}_t)$.
 - Existe un único $\mathbf{w}^*$ tal que $\nabla J(\mathbf{w}^*) = 0$.
 - $\mathcal{H}(\mathbf{w})$ es definido positivo para todo $\mathbf{w}$.
- 3 hipótesis*

Nuestro problema ahora es:

¿Cuál es el mejor α ?

Convergencia y optimización para problemas convexos

Modelo convexo con derivadas segunda continuas

- $\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \nabla J(\mathbf{w}_t)$.
- Existe un único $\mathbf{w}^*$ tal que $\nabla J(\mathbf{w}^*) = 0$.
- $\mathcal{H}(\mathbf{w})$ es definido positivo para todo $\mathbf{w}$.

Teorema de Taylor

$$\nabla J(\mathbf{w}_t) = \nabla J(\mathbf{w}^*) + \mathcal{H}(\tilde{\mathbf{w}}) \cdot (\mathbf{w}_t - \mathbf{w}^*)$$

para algún $\tilde{\mathbf{w}}$ en el segmento que une $\mathbf{w}_t$ y $\mathbf{w}^*$.

En AMI era el teorema del valor medio

$$f(x) = f(x_0) + f'(\tilde{x})(x - x_0), \text{ donde } \tilde{x} \in [x, x_0]$$

Convergencia y optimización para problemas convexos

Modelo convexo con derivadas segunda continuas

- $\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \nabla J(\mathbf{w}_t)$.
- Existe un único $\mathbf{w}^*$ tal que $\nabla J(\mathbf{w}^*) = 0$.
- $\mathcal{H}(\mathbf{w})$ es definido positivo para todo $\mathbf{w}$.

Teorema de Taylor

$$\nabla J(\mathbf{w}_t) = \nabla J(\mathbf{w}^*) + \mathcal{H}(\tilde{\mathbf{w}}) \cdot (\mathbf{w}_t - \mathbf{w}^*)$$

para algún $\tilde{\mathbf{w}}$ en el segmento que une $\mathbf{w}_t$ y $\mathbf{w}^*$.

Diagonalización ortogonal

Toda matriz real, cuadrada y simétrica puede escribirse como $H = Q^T \Lambda Q$ con una matriz de autovalores Λ diagonal y una de autovectores Q ortogonal $Q^T Q = Q Q^T = I$.

$$\nabla J(\omega_+) = Q^T \Lambda Q (\omega_+ - \omega^*)$$

Convergencia y optimización para problemas convexos

$$\nabla J(\mathbf{w}_t) = \underbrace{\nabla J(\mathbf{w}^*)}_{=0} + \mathcal{H}(\tilde{\mathbf{w}}) \cdot (\mathbf{w}_t - \mathbf{w}^*) = Q^T \Lambda Q \cdot (\mathbf{w}_t - \mathbf{w}^*)$$

con Q y Λ las matrices correspondientes a la diagonalización de $\mathcal{H}(\tilde{\mathbf{w}})$.

Convergencia y optimización para problemas convexos

$$\nabla J(\mathbf{w}_t) = \nabla J(\mathbf{w}^*) + \mathcal{H}(\tilde{\mathbf{w}}) \cdot (\mathbf{w}_t - \mathbf{w}^*) = Q^T \Lambda Q \cdot (\mathbf{w}_t - \mathbf{w}^*)$$

con Q y Λ las matrices correspondientes a la diagonalización de $\mathcal{H}(\tilde{\mathbf{w}})$.

Regresión lineal

Si $J(\mathbf{w}) = \frac{1}{n} \|\mathbf{X} \cdot \mathbf{w} - \mathbf{y}\|^2$, $\mathcal{H}(\tilde{\mathbf{w}}) = \frac{2}{n} \mathbf{X}^T \mathbf{X}$ no depende del valor de los parámetros (solo de los datos).

Convergencia y optimización para problemas convexos

$$\nabla J(\mathbf{w}_t) = \nabla J(\mathbf{w}^*) + \mathcal{H}(\tilde{\mathbf{w}}) \cdot (\mathbf{w}_t - \mathbf{w}^*) = Q^T \Lambda Q \cdot (\mathbf{w}_t - \mathbf{w}^*)$$

con Q y Λ las matrices correspondientes a la diagonalización de $\mathcal{H}(\tilde{\mathbf{w}})$.

Regresión lineal

Si $J(\mathbf{w}) = \frac{1}{n} \|\mathbf{X} \cdot \mathbf{w} - \mathbf{y}\|^2$, $\mathcal{H}(\tilde{\mathbf{w}}) = \frac{2}{n} \mathbf{X}^T \mathbf{X}$ no depende del valor de los parámetros (solo de los datos).

$$\begin{aligned} \mathbf{w}_{t+1} - \mathbf{w}^* &= \mathbf{w}_t - \mathbf{w}^* - \alpha \nabla J(\mathbf{w}_t) = (\mathbf{w}_t - \mathbf{w}^*) - \alpha Q^T \Lambda Q (\mathbf{w}_t - \mathbf{w}^*) \\ &\stackrel{\text{Factor común}}{\leftarrow} \underset{(\mathbf{w}_t - \mathbf{w}^*)}{=} (I - \alpha Q^T \Lambda Q) (\mathbf{w}_t - \mathbf{w}^*) \\ &= Q^T (I - \alpha \Lambda) Q (\mathbf{w}_t - \mathbf{w}^*) \end{aligned}$$

$\mathbf{w}_{t+1}$

$\mathbf{I} = Q^T Q$
saca Q^T a $\mathbf{I}$ y Q a der.

Convergencia y optimización para problemas convexos

$$\nabla J(\mathbf{w}_t) = \nabla J(\mathbf{w}^*) + \mathcal{H}(\tilde{\mathbf{w}}) \cdot (\mathbf{w}_t - \mathbf{w}^*) = Q^T \Lambda Q \cdot (\mathbf{w}_t - \mathbf{w}^*)$$

con Q y Λ las matrices correspondientes a la diagonalización de $\mathcal{H}(\tilde{\mathbf{w}})$.

Regresión lineal

Si $J(\mathbf{w}) = \frac{1}{n} \|\mathbf{X} \cdot \mathbf{w} - \mathbf{y}\|^2$, $\mathcal{H}(\tilde{\mathbf{w}}) = \frac{2}{n} \mathbf{X}^T \mathbf{X}$ no depende del valor de los parámetros (solo de los datos).

$$\begin{aligned}\mathbf{w}_{t+1} - \mathbf{w}^* &= \mathbf{w}_t - \mathbf{w}^* - \alpha \nabla J(\mathbf{w}_t) \\ &= (I - \alpha Q^T \Lambda Q) (\mathbf{w}_t - \mathbf{w}^*) \\ &= Q^T (I - \alpha \Lambda) Q (\mathbf{w}_t - \mathbf{w}^*)\end{aligned}$$

Defino $v_t = Q (\mathbf{w}_t - \mathbf{w}^*)$, luego:

$$(\mathbf{w}_{t+1} - \mathbf{w}^*) = \underset{\substack{\uparrow \\ \text{X} \times Q}}{Q^T} (I - \alpha \Lambda) v_t$$

$$\begin{aligned}v_{t+1} &= (I - \alpha \Lambda) v_t, & v_{t+1} &= Q Q^T (I - \alpha \Lambda) v_t \\ & & v_{t+1} &= (I - \alpha \Lambda) v_t\end{aligned}$$

Convergencia y optimización para problemas convexos

$$\nabla J(\mathbf{w}_t) = \nabla J(\mathbf{w}^*) + \mathcal{H}(\tilde{\mathbf{w}}) \cdot (\mathbf{w}_t - \mathbf{w}^*) = Q^T \Lambda Q \cdot (\mathbf{w}_t - \mathbf{w}^*)$$

con Q y Λ las matrices correspondientes a la diagonalización de $\mathcal{H}(\tilde{\mathbf{w}})$.

Regresión lineal

Si $J(\mathbf{w}) = \frac{1}{n} \|\mathbf{X} \cdot \mathbf{w} - \mathbf{y}\|^2$, $\mathcal{H}(\tilde{\mathbf{w}}) = \frac{2}{n} \mathbf{X}^T \mathbf{X}$ no depende del valor de los parámetros (solo de los datos).

$$\begin{aligned} \mathbf{w}_{t+1} - \mathbf{w}^* &= \mathbf{w}_t - \mathbf{w}^* - \alpha \nabla J(\mathbf{w}_t) \\ &= (I - \alpha Q^T \Lambda Q) (\mathbf{w}_t - \mathbf{w}^*) \\ &= Q^T (I - \alpha \Lambda) Q (\mathbf{w}_t - \mathbf{w}^*) \end{aligned}$$

Defino $v_t = Q (\mathbf{w}_t - \mathbf{w}^*)$, luego:

$$v_{t+1} = (I - \alpha \Lambda) v_t, \quad v_t = \overbrace{(I - \alpha \Lambda)^t}^{\text{matriz diagonal}} v_0 \rightarrow \text{Se parece a una serie geométrica}$$

$v_0 \rightarrow v_1 = (I - \alpha \Lambda) v_0 \rightarrow v_2 = (I - \alpha \Lambda)^2 v_0$ y así

Convergencia y optimización para problemas convexos

Condición y velocidad de convergencia

El GD convergerá si $|1 - \alpha\lambda_j| < 1$ para todo j y el learning rate óptimo estará asociado al criterio de peor caso:

$$\min_{\alpha} \max_j |1 - \alpha\lambda_j| \quad \text{s.t.} \quad |1 - \alpha\lambda_j| < 1 \quad \forall j$$

Sujeto a q' $|1 - \alpha\lambda_j| < 1$

Convergencia y optimización para problemas convexos

Condición y velocidad de convergencia

El GD convergerá si $|1 - \alpha\lambda_j| < 1$ para todo j y el learning rate óptimo estará asociado al criterio de peor caso:

El mínimo valor que toma $\max_j |1 - \alpha\lambda_j|$ para distintos valores de α → el máximo valor que toma $|1 - \alpha\lambda_j|$ para distintos valores de j

$$\min_{\alpha} \max_j |1 - \alpha\lambda_j| \quad \text{s.t.} \quad |1 - \alpha\lambda_j| < 1 \quad \forall j$$

Recordar que $\lambda_j > 0$ para todo j .

Condición de convergencia

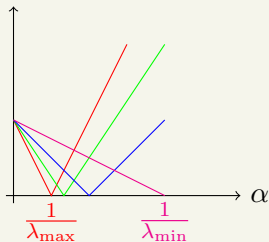
$|1 - \alpha\lambda_j| < 1$ para todo j equivale a pedir $\alpha < \frac{2}{\lambda_{\max}}$.

$$-1 < 1 - \alpha\lambda_j < 1 \rightarrow -2 < -\alpha\lambda_j < 0 \rightarrow 0 < \alpha\lambda_j < 2 \rightarrow \alpha < \frac{2}{\lambda_j} \quad \forall j$$

trivial, se cumple siempre

Convergencia y optimización para problemas convexos

$|1 - \alpha\lambda_j|$ El pico está donde $\alpha\lambda_j = 1$



Convergencia y optimización para problemas convexos

La línea negra es

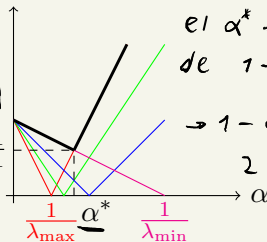
$$\max_j |1 - \alpha \lambda_j|$$

$$|1 - \alpha \lambda_j|$$

$1 - \alpha \lambda \rightarrow \text{decrece}$ $\alpha \lambda - 1 \rightarrow \text{crece}$

Esta altura es la velocidad de convergencia (el elemento de la diagonal de mayor valor

$$\leftarrow \frac{\kappa - 1}{\kappa + 1}$$



el α^* se da en la intersección de $1 - \alpha \lambda_{\min}$ y $\alpha \lambda_{\max} - 1$

$$\rightarrow 1 - \alpha^* \lambda_{\min} = \alpha^* \lambda_{\max} - 1$$

$$2 = (\lambda_{\min} + \lambda_{\max}) \alpha^*$$

$$\alpha^* = \frac{2}{\lambda_{\min} + \lambda_{\max}}$$

Velocidad de convergencia

El óptimo learning rate en este caso es $\alpha^* = \frac{2}{\lambda_{\max} + \lambda_{\min}}$ y su velocidad asociada $\left(\frac{\kappa - 1}{\kappa + 1}\right)^t$ depende del número de condición $\kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$.

Optimalidad

El óptimo no es el más grande convergente: $\alpha^* = \frac{2}{\lambda_{\max} + \lambda_{\min}} < \frac{2}{\lambda_{\max}}$

Outline

Vel de convergencia: En la recta $1 - \alpha \lambda_{\min}$ busca el valor para $\alpha = \alpha^k \rightarrow 1 - \frac{2}{\lambda_{\min} + \lambda_{\max}} \cdot \lambda_{\min} = \text{NFM} = \frac{\lambda_{\max} / \lambda_{\min} - 1}{\lambda_{\max} / \lambda_{\min} + 1}$

1 Introducción al problema de regresión

Tips para el TP: No tiene q' haber integrales en Y

2 Regresión Lineal

$$E[Y|X=x] = r(x)$$

Fórmula para no integrar en X:

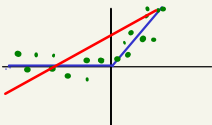
3 Gradiente Descendente

$$E[g(x)|X \in A] = \frac{E[g(x) \mathbb{1}\{X \in A\}]}{\mathbb{P}(X \in A)}$$

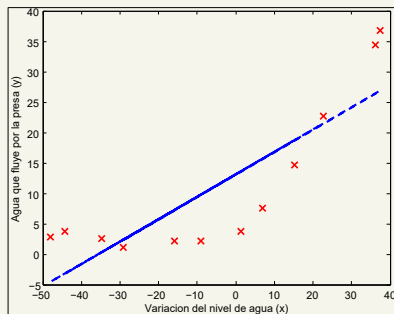
4 Regresión Polinómica

$$E[g(x) \cdot \mathbb{1}\{X \in A\}] = E[g(x)|X \in A] \cdot \mathbb{P}(X \in A)$$

???



¿Y si la complejidad lineal no alcanza?

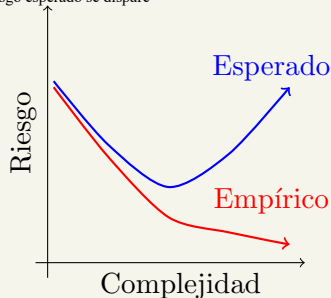
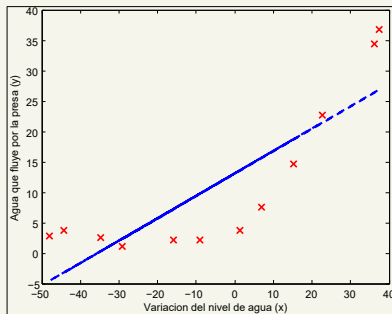


28/08 Seguimos acá

NO para de tomar mate este loco
Los riñones de acero

¿Y si la complejidad lineal no alcanza?

Agregando un grado crecemos en complejidad. Si no tenemos cuidado puede que el riesgo esperado se dispare

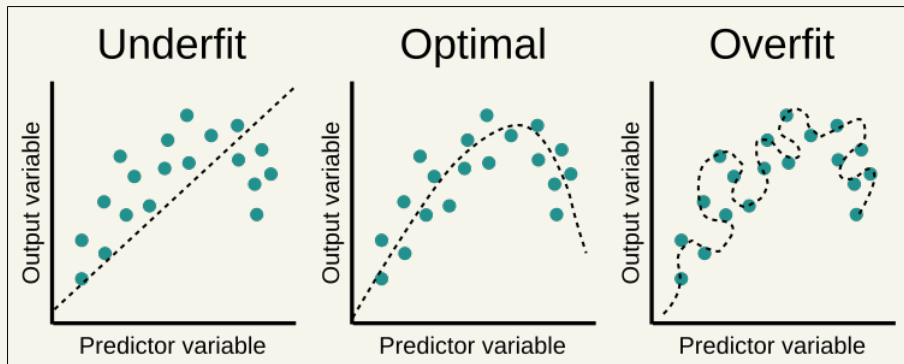


Regresión Polinómica

$$\mathbf{X} = \begin{pmatrix} 1 & X_{1,1} & X_{1,2} & X_{1,1}^2 & X_{1,2}^2 & X_{1,1}X_{1,2} \\ 1 & X_{2,1} & X_{2,2} & X_{2,1}^2 & X_{2,2}^2 & X_{2,1}X_{2,2} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & X_{n,1} & X_{n,2} & X_{n,1}^2 & X_{n,2}^2 & X_{n,1}X_{n,2} \end{pmatrix}$$

Compromiso Sesgo/Varianza

Riesgo esperado = riesgo empírico + gap de generalización



Si no puedo confiar en los datos de entrenamiento ¿Como procedo?

Conjuntos de datos

- Conjunto de entrenamiento (*train set*): Datos utilizados para minimizar el riesgo empírico. Sobre estos se produce el “aprendizaje”. Las variables definidas a partir de este conjunto se llaman parámetros.

¿Cómo sé si es mejor usar un polinomio de grado 4 o 5? Corro el algoritmo sobre el conjunto de validación y me fijo cuál da más error

- Conjunto de validación (*validation or development set*): Datos utilizados para comparar modelos. Las variables definidas a partir de este conjunto (o definidas previas al entrenamiento) se llaman hiperparámetros. Por ejemplo, el grado del polinomio sería un hiperparámetro. Cuántas iteraciones hago en el gradiente descendiente es un hiperparámetro.
- Conjunto de testeo (*test set*): Datos utilizados para evaluar la performance final del algoritmo. Su única función es presentar estimadores insesgados de las métricas de error y no es imprescindible.

Si la base de datos esta dividida, respetar la división!

Enfoque clásico: 60%/20%/20% - Típico para 1K, 10K muestras.

Big Data: Para 1M muestras, quizás alcanza con 98%/1%/1%.

Atacar el punto débil

¿Que conviene corregir? ¿Sesgo o varianza?

- **Avoidable bias:** Error de train - Error bayesiano
- **Generalization Gap:** Error de validación - Error de train

→ El riesgo empírico es demasiado grande, se aleja mucho del bayesiano. Underfitting. Solución: aumentar en complejidad, mucho más no hay

→ Entrené el algoritmo, lo corri sobre es set de validación y dio para el orto. Overfitting.
Necesito más datos, pero eso es más fácil decirlo que hacerlo

Los enfoques para resolver estos problemas son distintos. Empecemos por el problema de Overfitting

Atacar el punto débil

¿Que conviene corregir? ¿Sesgo o varianza?

- **Avoidable bias:** Error de train - Error bayesiano
- **Generalization Gap:** Error de validación - Error de train

Técnica Clásica de Regularización

Se agrega un término de penalización que perturba la optimización del riesgo empírico: *vector de parámetros*

$$\hat{y} = w \cdot x + b$$

$$J(\hat{\theta}) = \frac{1}{n} \sum_{i=1}^n [L_i(\theta)] + \lambda R(\theta)$$

$$\theta = (w, b)$$

en regresión lineal sería
 $L(\theta) = (y - \hat{y})^2$

Atacar el punto débil

¿Que conviene corregir? ¿Sesgo o varianza?

- **Avoidable bias:** Error de train - Error bayesiano
- **Generalization Gap:** Error de validación - Error de train

Técnica Clásica de Regularización

Se agrega un término de penalización que perturba la optimización del riesgo empírico:

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n L_i(\theta) + \lambda R(\theta)$$

Motivación: Error de generalización

El regularizador trata ser representativo del error de generalización:

$$\mathbb{E}[L(\theta)] = \frac{1}{n} \sum_{i=1}^n L_i(\theta) + \overbrace{\left(\mathbb{E}[L(\theta)] - \frac{1}{n} \sum_{i=1}^n L_i(\theta) \right)}^{\lambda \cdot R(\theta)}$$

Regresión Lineal Regularizada

Weight decay or L2 regularization

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n L_i(\theta) + \frac{\lambda}{n} \|\mathbf{w}\|^2 \rightarrow \frac{\partial \|\mathbf{w}\|^2}{\partial \mathbf{w}} = 2\mathbf{w}$$

Regresión Lineal Regularizada

Weight decay or L2 regularization

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n L_i(\theta) + \frac{\lambda}{n} \|\mathbf{w}\|^2 \rightarrow \frac{\partial \|\mathbf{w}\|^2}{\partial \mathbf{w}} = 2\mathbf{w}$$

Interpretación 1: Apagar parámetros

$w_j \approx 0$ simplifica la complejidad del modelo.

Considerar el caso del gradiente descendiente: definiendo el error de esta manera, el algoritmo tiende a penalizar valores grandes de w , el gradiente tiende a caer para el otro lado

Regresión Lineal Regularizada

Weight decay or L2 regularization

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n L_i(\theta) + \frac{\lambda}{n} \|\mathbf{w}\|^2 \rightarrow \frac{\partial \|\mathbf{w}\|^2}{\partial \mathbf{w}} = 2\mathbf{w}$$

Interpretación 1: Apagar parámetros

$w_j \approx 0$ simplifica la complejidad del modelo.

Interpretación 2: Disminuir el máximo valor de la función costo

$$\mathbb{E}[L(\theta)] - \frac{1}{n} \sum_{i=1}^n L_i(\theta) \leq \max_{\phi \in \Theta} L(\phi)$$

Validación: ¿Como elijo el λ ?

Set de Validación

Si tengo una buena cantidad de datos de validación, elijo el λ con menor error de validación.

Por ejemplo:

```
for lambda in [conjunto de lambdas]:
```

```
    fit(X, Y, lambda)
```

```
    ECM(lambda) = np.mean((y_validacion - predict(X_validacion))**2)
```

```
lambda_final = agr min ECM(lambda)
```

Validación: ¿Como elijo el λ ?

Set de Validación

Si tengo una buena cantidad de datos de validación, elijo el λ con menor error de validación.

Leave-one-out cross-validation (LOOCV)

Si tengo pocos datos no puedo tener un conjunto de datos de validación suficientemente rico. Entonces entreno con todas las muestras menos una y valido con la última. Luego repito esto con cada muestra y promedio.

ganas en datos, porque necesitas menos, pero laburás más el algoritmo

Validación: ¿Como elijo el λ ?

Set de Validación

Si tengo una buena cantidad de datos de validación, elijo el λ con menor error de validación.

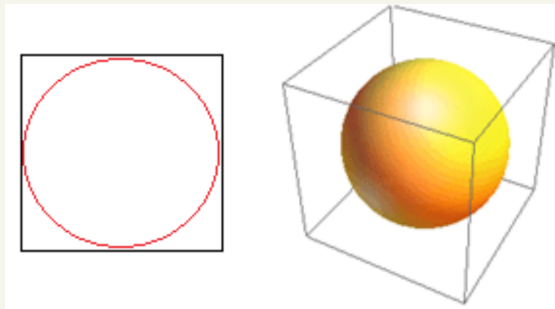
Leave-one-out cross-validation (LOOCV)

Si tengo pocos datos no puedo tener un conjunto de datos de validación suficientemente rico. Entonces entreno con todas las muestras menos una y valido con la última. Luego repito esto con cada muestra y promedio.

K-Fold → Punto medio entre LOOCV y usar un solo set de validación

Separo en K subgrupos de $\frac{n}{K}$ muestras cada uno. Entreno con $K - 1$ grupos y testeo con el último. Luego repito esto con cada grupo y promedio.

La maldición de la dimensionalidad



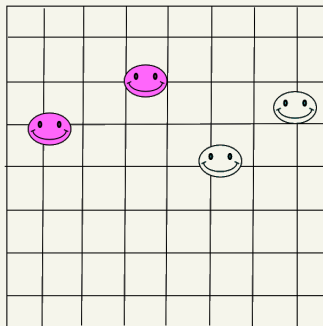
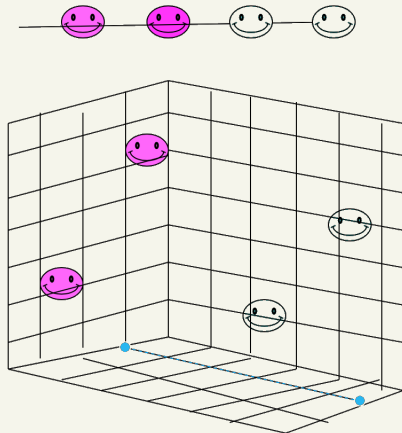
- 2d: $\frac{\pi r^2}{(2r)^2} \approx 78.5\%$
- 3d: $\frac{\frac{4}{3}\pi r^3}{(2r)^3} \approx 52.3\%$
- 10d: $\frac{\frac{r^{10}}{5!}\pi^5}{(2r)^{10}} \approx 0.25\%$

En grandes dimensiones:

- Los puntos están muy lejos.
- Las estructuras son muy sparse.
- La distancia euclídea no es buena métrica.
- La “necesidad” de muestras crece exponencialmente con la dimensión.

La maldición de la dimensionalidad

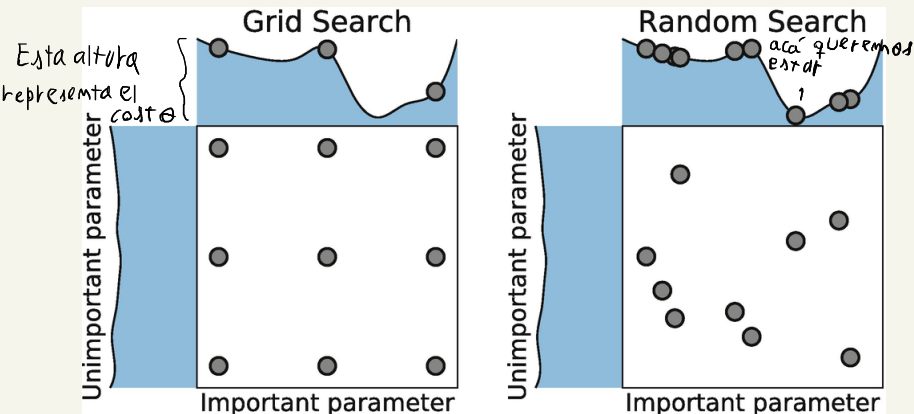
La maldición aplica a los hiperparámetros



**La necesidad de pruebas
crece exponencialmente
con la cantidad de
hiperparámetros!**

Búsqueda aleatoria

No todos los hiperparámetros son igual de importantes



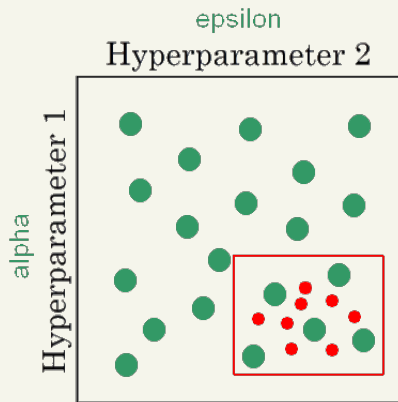
Random search nos permite variar muchas veces todos los parámetros.

No es fácil saber a priori qué hiperparámetro importa

Búsqueda aleatoria

Hacerlo por etapas permite aprovechar más las simulaciones.

Aguanten estos dibujitos



Simulo unos pocos puntos, veo donde está andando mejor y vuelvo a simular dentro de ese entorno.